

Spec Kit

A Beginner's Tutorial

Build high-quality software faster with Spec-Driven Development

What is Spec Kit?

Spec Kit is an **open-source toolkit** from GitHub that helps you build software using a method called **Spec-Driven Development (SDD)**. Instead of diving straight into writing code, you first write a clear *specification* (a document describing *what* you want to build and *why*). An AI coding agent then reads that spec and handles the implementation for you.

Think of it like an architect drawing blueprints before builders start construction. The spec is your blueprint — precise, detailed, and agreed upon *before* any code is written.

Why should beginners care?

- **Fewer mistakes.** Clear requirements mean the AI builds exactly what you intended.
- **Easier to review.** You review a short English spec instead of hundreds of lines of code.
- **Works with many AI agents.** GitHub Copilot, Claude Code, Gemini CLI, and 30+ others are supported.
- **Scalable habit.** The same discipline used by professional engineering teams.

Before You Start

Make sure you have the following tools installed:

Python 3.11+	Download from python.org. Required to run Spec Kit.
uv (recommended)	A fast Python package manager. Install: <code>pip install uv</code>
Git	Version control. Download from git-scm.com.
An AI coding agent	e.g. GitHub Copilot, Claude Code, or Gemini CLI.

Step 1 — Install Specify CLI

The **Specify CLI** is the command-line tool that comes with Spec Kit. Install it directly from GitHub (do *not* use PyPI — those packages are unrelated):

Option A — Persistent install (recommended):

```
# Replace vX.Y.Z with the latest release tag from
# https://github.com/github/spec-kit/releases
uv tool install specify-cli \
  --from git+https://github.com/github/spec-kit.git@vX.Y.Z
```

Option B — Run once without installing:

```
uvx --from git+https://github.com/github/spec-kit.git@vX.Y.Z \
  specify init my-project
```

Verify the installation:

```
specify version
```

You should see the version number printed in the terminal. If you get a 'command not found' error, make sure **uv**'s bin directory is in your PATH.

Step 2 — Create a New Project

Navigate to the folder where you want your project to live and run:

```
# Create a new folder called 'my-app' and set everything up
specify init my-app --integration copilot

# Or initialise inside an existing folder
cd existing-project
specify init . --integration copilot
```

Replace `copilot` with your agent: `claude`, `gemini`, `codex`, etc. The CLI will create a `.specify/` folder with templates and scripts.

What gets created:

```
.specify/
  memory/
    constitution.md  <- project principles (created in Step 3)
  scripts/          <- helper shell scripts
  specs/            <- your feature specs live here
  templates/        <- spec, plan and task templates
```

The Spec-Driven Workflow at a Glance

Every feature you build follows the same 6-step loop inside your AI coding agent.

1	/speckit.constitution Set your project's principles — coding standards, testing rules, UX guidelines.
2	/speckit.specify Describe WHAT you want to build. Focus on features and user needs, not technology.
3	/speckit.clarify (optional but recommended) Let the agent ask clarifying questions to fill any gaps in your spec.
4	/speckit.plan Provide the tech stack. The agent writes a detailed technical implementation plan.
5	/speckit.tasks Break the plan into an ordered, dependency-aware task list.
6	/speckit.implement The agent executes every task and writes the actual code.

Detailed Walkthrough

Step 1 — Establish Principles

Open your AI agent inside the project folder and run:

```
/speckit.constitution Create principles focused on code quality,
testing standards, user experience, and performance.
```

This creates `.specify/memory/constitution.md` — a file the agent will reference throughout all future steps to keep decisions consistent.

Step 2 — Write Your Specification

Describe your feature in plain English. Be specific about *what* users can do but don't mention the tech stack yet:

```
/speckit.specify Build a photo album app where users can:
- Create albums grouped by date
- Drag and drop photos between albums
- Preview photos in a tile grid
Albums cannot be nested inside other albums.
```

The agent creates a `spec.md` file with user stories and acceptance criteria.

Step 3 — Clarify (Optional)

Run `/speckit.clarify` to let the agent ask follow-up questions. This catches ambiguities early and saves rework later.

Step 4 — Create the Technical Plan

Now introduce the tech stack:

```
/speckit.plan Use React with Vite. Store metadata in a local
SQLite database. Keep dependencies minimal.
```

Output: `plan.md`, `data-model.md`, API contracts, and more.

Step 5 — Generate Tasks

```
/speckit.tasks
```

Produces a `tasks.md` with tasks ordered by dependency. Tasks that can run in parallel are marked [P].

Step 6 — Implement

```
/speckit.implement
```

The agent validates all prerequisite files, then executes every task in order, running CLI commands (npm, python, etc.) as needed. Make sure the required tools are installed on your machine.

Quick Command Reference

Command	What it does
<code>/speckit.constitution</code>	Create or update project principles
<code>/speckit.specify</code>	Define requirements and user stories
<code>/speckit.clarify</code>	Structured Q&A; to fill spec gaps
<code>/speckit.plan</code>	Generate technical implementation plan
<code>/speckit.analyze</code>	Cross-check artifacts before implementation
<code>/speckit.tasks</code>	Break plan into actionable task list
<code>/speckit.implement</code>	Execute all tasks and write the code
<code>/speckit.checklist</code>	Generate quality checklist from requirements

Specify CLI Commands

Command	What it does
<code>specify init</code>	Bootstrap a new project
<code>specify init . --integration X</code>	Initialise in the current directory
<code>specify check</code>	Verify prerequisites are installed
<code>specify version</code>	Print the installed version
<code>specify extension search</code>	Browse community extensions
<code>specify extension add</code>	Install an extension
<code>specify preset search</code>	Browse community presets
<code>specify preset add</code>	Install a preset
<code>specify integration list</code>	List all supported AI agent integrations

Customising Spec Kit

Once you are comfortable with the basics, Spec Kit can be extended in two ways:

Extensions	Presets
Add <i>new</i> commands and capabilities. Examples: Jira sync, post-implementation code review, security audit, GitHub Issues integration.	<i>Override</i> existing templates to match your team's standards. Examples: compliance spec formats, different languages, Agile terminology.
<code>specify extension search</code> <code>specify extension add</code>	<code>specify preset search</code> <code>specify preset add</code>

Tips for Beginners

- **Focus on the 'what', not the 'how'** — during `/speckit.specify`, describe user needs and features. Save technology choices for `/speckit.plan`.
- **Be explicit** — vague specs produce vague code. The more detail you provide, the better the output.
- **Use `/speckit.clarify`** — don't skip this step on your first project. It exposes assumptions you didn't know you were making.
- **Pin a release tag** — use a specific version (e.g., `@v0.8.4`) to get predictable results rather than installing from 'main'.
- **Check the README first** — if something doesn't work, the latest instructions are at github.com/github/spec-kit.

Learn More

GitHub Repository	github.com/github/spec-kit
Official Docs	github.github.io/spec-kit/
Detailed Guide	github.com/github/spec-kit/blob/main/spec-driven.md
Video Overview	youtube.com/watch?v=a9eR1xsfvHg
Community Ext.	speckit-community.github.io/extensions/
Issues / Support	github.com/github/spec-kit/issues

Licensed under the MIT License. Spec Kit is maintained by GitHub.